

Why the marginal HMC blur update in *bsbd* loses to Gibbs on ESS/s

Code review, numerical diagnosis, and an adjoint gradient

Prepared for Andrew Holbrook

github.com/guillerminasenn/bsbd (commit c42ccb6)

for Senn, Tjelmeland, Glatt-Holtz, Walker & Holbrook, *Bayesian Semi-Blind Deconvolution at Scale*, arXiv:2601.09677v1

29 September 2026

Summary

The paper reports that the marginal (collapsed) HMC blur update costs about $25\times$ a Gibbs iteration (13s vs. 0.47s on the 480×100 real example) while its ESS for the blur is only about $4\times$ that of Gibbs (133 vs. 32 out of 55k draws, Table 1), so Gibbs wins on ESS per second. Three things in the implementation account for this, in decreasing order of impact.

1. **Bug — the gradient of the potential is wrong.** In `gradient_potential_Fourier_domain` the vector $\mathbf{v} = \Sigma_{d|\omega}^{-1}(\mathbf{d} - \mathbf{W}\boldsymbol{\mu}_c)$ is silently overwritten by in-place FFTs before it is used, so the sum-of-squares part of the gradient is garbage (150–500% relative error against finite differences; the prior and log-determinant parts are correct). The sampler remains exact — Metropolis uses the correct potential — but the leapfrog integrator no longer conserves energy, so the adapted step size collapses by $\sim 300\times$, trajectories go almost nowhere, and ESS drops $\sim 25\times$. A one-line fix (`.copy()`).
2. **Algorithmic — the k gradient coordinates are computed one direction at a time (forward mode).** Section 4.2.2 of the paper derives $\partial U / \partial w_i$ per coordinate, and the code vectorises that over the k coordinates with (k, n_v, n_h) , (k, m, m) and (k, n_v, n_v) tensors. The true per-leapfrog cost is therefore $\mathcal{O}(k(n \log n + m^3 + n(n_v + n_h)))$ — a factor $k = 126$ above the complexity stated on p. 23 — plus ~ 1 GB of temporaries per gradient. Every term is linear in the direction eigenvalue ε_i , so all k coordinates follow from one n -sized reduction and one length- n_v FFT (reverse mode). Section 3 derives this; the implementation (Appendix B) reproduces the fixed gradient to 10^{-14} and cuts one gradient at the paper’s problem size from 3.2s to 39 ms.
3. **Bug — the mass matrix is the identity, not Σ_ω^{-1} .** `_create_momentum_object` reads the `precondition` option from the wrong dictionary level, so it is always `None`; had it been read correctly the code would have crashed because the stored precision matrix is a `csr_matrix`. The preconditioning described in Section 4.2 never ran.

With the two bugs fixed and the adjoint gradient, HMC’s ESS per second exceeds Gibbs’ by $\sim 3\times$ on a 2160-node test problem (Section 5), and the advantage should grow with problem size because the removed overhead is proportional to k . None of this invalidates the paper’s posterior results — only the efficiency comparison.

1 The gradient is wrong: in-place FFTs on a view of p

`src/mcmc/update_blur_image/collapsedhmc/cyclic/gradient_potential.py`, line 219, prepares the inputs of the sum-of-squares gradient:

```
### Compute P_R_ch_starT and R_cv_star_WOt in the time domain, used to compute the gradient of SS
if _c.reflectivity_constraints['constr']:
    P = p.reshape((nv, nh), order='F') # <-- a VIEW of p (p is (n,1), contiguous)
    R_ch_star = _c.reflectivity_constraints['Rh_star']
    P_R_ch_starT = P @ (R_ch_star.T)
    ...
grad_ss = gradient_SS_vectorized(sampler, i, p, P, eigenvalues_D1, ...)
```

`derivatives.py::gradient_SS_vectorized` then transforms P in place (NumPy ≥ 2.0 accepts `out=` in the FFT functions; the repository pins numpy 2.4.1):

```
np.fft.fft(P, axis=1, out=P) # Overwrite P with its FFT <-- also overwrites p
np.multiply(P, fft_base_Rch[None, :], out=P)
np.fft.ifft(P, axis=1, out=P)
np.fft.fft(P, axis=0, out=P)
P_after_D1_all = np.fft.ifft(P[None, :, :] * fft_base_D1_j_array[:, :, None], axis=1, out=temp_array_knvnh)
d_SS_1_part1_vectorized[unconstrained_indices] = (P_after_D1_all.reshape(k, -1, order='F')) @ p # p is now corrupted
...
d_SS_1_part2_vectorized[unconstrained_indices] = products_resaped @ p # corrupted
d_SS_2_vectorized[unconstrained_indices] = (-2 * gamma_matrix_unconstrained_indexes @ p) # corrupted
```

The aliasing in isolation:

```
>>> p = np.arange(12, dtype=complex).reshape(-1, 1); P = p.reshape((4, 3), order='F')
>>> np.shares_memory(P, p)
True
>>> np.fft.fft(P, axis=1, out=P); p[4:].ravel()
array([12.+0.j, 15.+0.j, 18.+0.j, 21.+0.j]) # p no longer holds Sigma^-1 dbar
```

Evidence. Central finite differences of the repository’s own `potential()`, split into its three parts, on two simulated problems (full well column, endpoint-constrained blur, hyper-parameters as in `notebooks/run.ipynb`). Entries are $\max |\text{analytic} - \text{FD}| / \max |\text{FD}|$ over the free blur coordinates.

problem	prior grad.	$\partial \log \Sigma_{d \omega} $ ($\mathbf{A}, \mathbf{Y}$ terms)	$\partial(\text{sum of sq.})$	total
$n = 24$ (12×2 , $k = 6$, $m = 12$), as shipped	1.5×10^{-10}	6.5×10^{-9}	4.97 (wrong signs)	5.14
$n = 432$ (36×12 , $k = 12$, $m = 24$), as shipped	8.9×10^{-11}	5.9×10^{-8}	1.51	6.12
$n = 24$, with $P = P.\text{copy}()$	1.5×10^{-10}	6.5×10^{-9}	2.7×10^{-9}	6.8×10^{-10}
$n = 432$, with $P = P.\text{copy}()$	8.9×10^{-11}	5.9×10^{-8}	1.3×10^{-7}	3.3×10^{-7}

The potential itself was also checked against dense linear algebra (explicit $\Sigma_{d|\omega} = \mathbf{W} \tilde{\Sigma}_c \mathbf{W}^\top + \Sigma_d$): log-determinant and quadratic form agree to 10^{-15} , so the target is right and only the force is wrong.

Consequence. Leapfrog with any position-dependent force field is still volume preserving and reversible, so accept/reject with the exact Hamiltonian keeps the chain exact — the posteriors in the paper stand. But the proposal is no longer energy conserving: to reach the target acceptance rate the adaptation drives ε down by two to three orders of magnitude ($0.08 \rightarrow 0.0003$ on the $n = 432$ problem; $0.063 \rightarrow 0.0001$ on $n = 2160$), and with $L = 10$ the trajectory covers almost none of the posterior. The run stored in the repository’s `estimations/` folder shows the same signature (ε adapted $0.008 \rightarrow 0.001$, acceptance 0.37, on a problem with only 4 free blur coordinates). This is the dominant reason the HMC ESS in Table 1 is only 133 out of 55k for a 126-dimensional, nearly Gaussian marginal.

2 Per-coordinate (forward-mode) gradient: a hidden factor of k

Sections 4.2.2.1–4.2.2.2 evaluate the gradient one coordinate at a time: “each coordinate of $\partial \log |\Sigma_{d|\omega}| / \partial \mathbf{w}^*$ is obtained ... Therefore (65) is evaluated in $\mathcal{O}(n \log n + m^3)$ ” and “the cost of evaluating (67) is bounded by $\mathcal{O}(n(\log n + n_v + n_h))$ ”. Summed over the k free coordinates this is $\mathcal{O}(k(n \log n + m^3 + n(n_v + n_h)))$ per leapfrog step, not the $\mathcal{O}(k^2 + n \log n + m^3 + n(n_h + n_v))$ stated on p. 23. The implementation does exactly the per-coordinate computation, “vectorised” by stacking the k directions into leading array dimensions:

- three preallocated (k , n_v , n_h) complex temporaries (95 MB each at the paper’s size) through ~ 15 element-wise passes;
- `scipy.fft.ifft2(eigenvalues_dZ, axes=(1,2))`: k two-dimensional FFTs of size n ;
- a Python loop for `idx in range(k)`: `subset_base_dZ[idx] = subset_ABA(...)` into a (k, m, m) tensor;
- `dY = -(L.T) @ subset_base_dZ @ L` and `np.trace(inv_Y @ dY, axis1=1, axis2=2)`: three batched $m \times m$ products per coordinate, i.e. $3km^3 \approx 27$ GFLOP per gradient at $k = 124$, $m = 330$ (the last one only to read a trace);
- `Dh_star_array = R_cv_star_W0t[idx, :]`: a (k, n_v, n_v) tensor (229 MB) plus its transpose, then a batched $(n_v \times n_v)(n_v \times n_h)$ product per coordinate (5.7 GFLOP).

Measured breakdown of one gradient at the paper’s size ($n_v = 480$, $n_h = 100$, $k = 126$, $m = 330$; single-threaded OpenBLAS in a 2-vCPU sandbox):

component	time	share
gradient_SS_vectorized ((k, n_v, n_v) tensors + batched matmul)	1251 ms	39%
element-wise ops on (k, n_v, n_h) arrays + $\mathbf{L}^\top (\partial \mathbf{Z}) \mathbf{L}$ sandwich + $\text{inv_Y} \odot \mathbf{dY}$	1640 ms	52%
batched ifft2 over k coordinates	122 ms	4%
subset_ABA loop over k coordinates	76 ms	2%
everything that is $\mathcal{O}(n \log n + m^3)$ once (eigenvalues, $\mathbf{Y}^{-1}$, $\mathbf{S}^{-1}$, $\mathbf{v}$)	~ 70 ms	2%
total per gradient ($\times 11$ per HMC update with $L = 10$)	3171 ms	

Section 3 shows that all of this collapses to $\mathcal{O}(n \log n + m^3 + n_v^2 n_h)$ per gradient, with no k -fold arrays. Verification and timing of the resulting implementation (Appendix B; harness in Appendix C):

problem $(n_v \times n_h, k, m)$	adj. vs. fixed repo	adj. vs. FD	repo grad.	adjoint grad.	repo pot.	adjoint pot.
$12 \times 2, 6, 12$ ($n = 24$)	3.6×10^{-16}	8.1×10^{-10}	1.6 ms	1.1 ms	1.5 ms	0.8 ms
$36 \times 12, 12, 24$ ($n = 432$)	2.0×10^{-14}	2.7×10^{-7}	1.6 ms	0.9 ms	1.3 ms	0.6 ms
$90 \times 24, 30, 60$ ($n = 2160$)	1.0×10^{-14}	5.2×10^{-8}	12.1 ms	3.1 ms	5.2 ms	3.0 ms
$480 \times 100, 126, 330$ ($n = 48000$, paper)	1.1×10^{-14}	n/a	3212 ms	39 ms	339 ms	50 ms

At the paper’s size one HMC blur update (11 gradients + 2 potentials, $L = 10$) goes from ~ 36 s to ~ 0.53 s on this machine (68 $\times$). For reference the Gibbs blur update takes 41 ms and the Gibbs image update 382 ms here (Section 6 explains the latter).

3 Derivation of the adjoint gradient

3.1 Setting and conventions

All lattice vectors are column-major vectorisations, $\mathbf{d} = \text{vec}(\mathbf{D})$ with $\mathbf{D} \in \mathbb{R}^{n_v \times n_h}$, $n = n_v n_h$. A circulant with *base* (first row) $\mathbf{c} \in \mathbb{R}^{n_v}$ is $[\text{circ}(\mathbf{c})]_{pq} = c_{(q-p) \bmod n_v}$, and a BCCB with base $\mathbf{\Pi} \in \mathbb{R}^{n_v \times n_h}$ has block (i, j) equal to $\text{circ}(\mathbf{\Pi}[:, (j-i) \bmod n_h])$. For such a matrix $\mathbf{C}$ write $\lambda_C := \text{DFT}_2(\text{base}) \in \mathbb{C}^{n_v \times n_h}$, exactly the arrays the code calls **eigenvalues_***. The only facts used below are

$$\lambda_{MN} = \lambda_M \odot \lambda_N, \quad \lambda_{M^\top} = \overline{\lambda_M}, \quad \text{base}(\mathbf{C}) = \text{IDFT}_2(\lambda_C), \quad \lambda_{\mathbf{H} \otimes \mathbf{C}}(f_v, f_h) = \lambda_H(f_h) \lambda_C(f_v), \quad (1)$$

where $\text{IDFT}_2(\lambda)(\delta_v, \delta_h) = \frac{1}{n} \sum_f \lambda(f) e^{2\pi i(f_v \delta_v / n_v + f_h \delta_h / n_h)}$. (Whether one calls λ or $\bar{\lambda}$ “the” eigenvalues is immaterial: every formula is used consistently in the code’s convention, and the end result was checked against finite differences.)

The blur enters through $\mathbf{W}_0 = \text{circ}(\mathbf{P}\mathbf{w}^*)$ and $\mathbf{W} = \mathbf{I}_{n_h} \otimes \mathbf{W}_0$. For the i -th free coordinate, $\mathbf{P}\mathbf{e}_i = \mathbf{e}_{r_i}$ (a unit vector at position r_i ; **row_indexes** in the code), so

$$\mathbf{E}_i := \frac{\partial \mathbf{W}_0}{\partial w_i} = \text{circ}(\mathbf{e}_{r_i}), \quad \mathcal{E}_i := \frac{\partial \mathbf{W}}{\partial w_i} = \mathbf{I}_{n_h} \otimes \mathbf{E}_i, \quad \lambda_{\mathcal{E}_i}(f_v, f_h) = \varepsilon_i(f_v) := e^{-2\pi i f_v r_i / n_v}. \quad (2)$$

$\mathbf{E}_i$ is a cyclic shift: $[\mathbf{E}_i \mathbf{X}]_{p,:} = \mathbf{X}_{(p+r_i) \bmod n_v, :}$, and for a circulant $\text{circ}(\mathbf{b})$ the product $\mathbf{E}_i \text{circ}(\mathbf{b}) = \text{circ}(\text{roll}(\mathbf{b}, r_i))$ has base $b_{(\cdot - r_i)}$. Everything that follows is a consequence of one observation: *every per-coordinate quantity in Section 4.2.2 of the paper is a linear (or conjugate-linear) functional of ε_i , and ε_i depends on i only through the shift r_i .*

Shift-evaluation lemma. For any $\mathbf{G} \in \mathbb{C}^{n_v \times n_h}$, with $g(f_v) := \sum_{f_h} \mathbf{G}(f_v, f_h)$,

$$\begin{aligned} \sum_{f_v, f_h} \mathbf{G}(f_v, f_h) \varepsilon_i(f_v) &= \sum_{f_v} g(f_v) e^{-2\pi i f_v r_i / n_v} = [\text{DFT}_{n_v}(g)](r_i), \\ \sum_f \left(\mathbf{G} \varepsilon_i + \overline{\mathbf{G} \varepsilon_i} \right) &= 2 \text{Re} [\text{DFT}_{n_v}(g)](r_i). \end{aligned} \quad (3)$$

Hence the functional can be evaluated for *all* k free coordinates at once with one $\mathcal{O}(n)$ reduction, one length- n_v FFT and a gather at the k shifts $\{r_i\}$, instead of k separate $\mathcal{O}(n)$ contractions with k different $\mathbf{G} \odot \varepsilon_i$ arrays.

3.2 The potential and what is shared with it

With the paper's notation (eqs. 46–60), $\mathbf{A} = \mathbf{W}\Sigma_c\mathbf{W}^\top + \Sigma_d$ (BCCB), $\mathbf{B} = \mathbf{A}^{-1}\mathbf{W}\Sigma_c$ (BCCB), $\tilde{\Sigma}_c = \Sigma_c - \Sigma_c\mathbf{A}_c^\top(\mathbf{A}_c\Sigma_c\mathbf{A}_c^\top)^{-1}\mathbf{A}_c\Sigma_c$, $\Sigma_{d|\omega} = \mathbf{W}\tilde{\Sigma}_c\mathbf{W}^\top + \Sigma_d$, and

$$\log |\Sigma_{d|\omega}| = \log |\mathbf{Y}| + \sum_f \log \lambda_A(f), \quad \mathbf{Y} = \mathbf{I}_m - \mathbf{L}^\top \mathbf{A}_c \tilde{\mathbf{Z}} \mathbf{A}_c^\top \mathbf{L}, \quad \tilde{\mathbf{Z}} := \sigma_c^2 \mathbf{R}_c \mathbf{W}^\top \mathbf{A}^{-1} \mathbf{W} \mathbf{R}_c, \quad (4)$$

$$\Sigma_{d|\omega}^{-1} = \mathbf{A}^{-1} + \mathbf{B} \mathbf{A}_c^\top \mathbf{S}^{-1} \mathbf{A}_c \mathbf{B}^\top, \quad \mathbf{S} = \mathbf{A}_c (\Sigma_c - \mathbf{Z}) \mathbf{A}_c^\top. \quad (5)$$

Here $\mathbf{L}\mathbf{L}^\top = (\mathbf{A}_c \mathbf{R}_c \mathbf{A}_c^\top)^{-1}$ as in the code (correlation rather than covariance, whence the scaling $\tilde{\mathbf{Z}} = \mathbf{Z}/\sigma_c^2$ in (4)), and $\lambda_A = \sigma_c^2 |\lambda_W|^2 \lambda_{R_c} + \sigma_d^2 \lambda_{R_d}$ is real and positive. Because the m exactly observed pixels lie in one column of the lattice, $\mathbf{A}_c \mathbf{C} \mathbf{A}_c^\top$ for any BCCB $\mathbf{C}$ is a submatrix of its *diagonal block*: $[\mathbf{A}_c \mathbf{C} \mathbf{A}_c^\top]_{ab} = c((\rho_b - \rho_a) \bmod n_v, 0)$ where ρ_a are the well rows and $c(\cdot, 0)$ the first column of the base array. With $\bar{\mathbf{d}} = \mathbf{d} - \mathbf{W}\mu_c$,

$$\mathbf{v} := \Sigma_{d|\omega}^{-1} \bar{\mathbf{d}} = \mathbf{A}^{-1} \bar{\mathbf{d}} + \mathbf{B} \mathbf{A}_c^\top \mathbf{S}^{-1} \mathbf{q}, \quad \mathbf{q} := \mathbf{A}_c \mathbf{B}^\top \bar{\mathbf{d}}, \quad (6)$$

which costs a handful of FFTs (each BCCB mat-vec is $\text{IDFT}_2(\lambda \odot \text{DFT}_2(\cdot))$, a gather/scatter on the well rows, and one $m \times m$ Cholesky solve. $U = -\log p(\omega) + \frac{1}{2} \log |\Sigma_{d|\omega}| + \frac{1}{2} \bar{\mathbf{d}}^\top \mathbf{v}$ is thus $\mathcal{O}(n \log n + m^3)$, and $\mathbf{Y}$, $\mathbf{S}$, $\mathbf{v}$, $\mathbf{q}$ are reused by the gradient below (the shipped code recomputes them in `potential()` and builds a dense $m \times n$ matrix for $\mathbf{q}$).

3.3 Gradient of the log-determinant

By Jacobi's formula, $\partial \log |\Sigma_{d|\omega}| / \partial w_i = \text{tr}(\mathbf{A}^{-1} \partial_i \mathbf{A}) + \text{tr}(\mathbf{Y}^{-1} \partial_i \mathbf{Y})$ (paper's eq. 65). The adjoint variables are $\mathbf{A}^{-1}$ and $\mathbf{Y}^{-1}$; the point is to contract them with $\partial_i \mathbf{A}$, $\partial_i \mathbf{Y}$ *without forming* $\partial_i \mathbf{A}$, $\partial_i \mathbf{Y}$ for each i .

Term $\text{tr}(\mathbf{A}^{-1} \partial_i \mathbf{A})$. From $\partial_i \mathbf{A} = \sigma_c^2 (\mathcal{E}_i \mathbf{R}_c \mathbf{W}^\top + \mathbf{W} \mathbf{R}_c \mathcal{E}_i^\top)$ and (1)–(2),

$$\lambda_{\partial_i \mathbf{A}} = \sigma_c^2 (\varepsilon_i \mathbf{G}_1 + \overline{\varepsilon_i \mathbf{G}_1}), \quad \mathbf{G}_1 := \lambda_{R_c} \odot \overline{\lambda_W}, \quad (7)$$

so, since the trace of a BCCB is the sum of its eigenvalues and λ_A is real, the lemma gives

$$\boxed{\text{tr}(\mathbf{A}^{-1} \partial_i \mathbf{A}) = 2\sigma_c^2 \text{Re}[\text{DFT}_{n_v}(h_1)](r_i), \quad h_1(f_v) = \sum_{f_h} \lambda_A^{-1}(f) \mathbf{G}_1(f)} \quad (8)$$

for all i at once: one $\mathcal{O}(n)$ reduction and one FFT of length n_v .

Term $\text{tr}(\mathbf{Y}^{-1} \partial_i \mathbf{Y})$. This is where the shipped code spends $3km^3$ flops and k IFFT2s. From (4), $\partial_i \mathbf{Y} = -\mathbf{L}^\top \mathbf{A}_c (\partial_i \tilde{\mathbf{Z}}) \mathbf{A}_c^\top \mathbf{L}$, and by cyclicity of the trace

$$\text{tr}(\mathbf{Y}^{-1} \partial_i \mathbf{Y}) = -\text{tr}(\mathbf{M} \mathbf{A}_c (\partial_i \tilde{\mathbf{Z}}) \mathbf{A}_c^\top) = -\langle \mathbf{M}, \mathbf{A}_c (\partial_i \tilde{\mathbf{Z}}) \mathbf{A}_c^\top \rangle_F, \quad \mathbf{M} := \mathbf{L} \mathbf{Y}^{-1} \mathbf{L}^\top \text{ (} m \times m, \text{ symmetric)}, \quad (9)$$

where $\langle \cdot, \cdot \rangle_F$ is the Frobenius inner product and we used that $\partial_i \tilde{\mathbf{Z}}$ is symmetric. $\mathbf{M}$ is formed *once*, $\mathcal{O}(m^3)$. Now use the diagonal-block structure: $[\mathbf{A}_c (\partial_i \tilde{\mathbf{Z}}) \mathbf{A}_c^\top]_{ab} = \tilde{z}_i((\rho_b - \rho_a) \bmod n_v)$ with $\tilde{z}_i(\delta) := \text{base}(\partial_i \tilde{\mathbf{Z}})(\delta, 0)$. Grouping the m^2 entries of $\mathbf{M}$ by lag,

$$\langle \mathbf{M}, \mathbf{A}_c (\partial_i \tilde{\mathbf{Z}}) \mathbf{A}_c^\top \rangle_F = \sum_{\delta=0}^{n_v-1} \mu(\delta) \tilde{z}_i(\delta), \quad \mu(\delta) := \sum_{a,b: (\rho_b - \rho_a) \bmod n_v = \delta} M_{ab}, \quad (10)$$

an $\mathcal{O}(m^2)$ histogram (`np.add.at` in the code) that is even, $\mu(\delta) = \mu(-\delta)$, because $\mathbf{M}$ is symmetric. Inserting $\tilde{z}_i(\delta) = \text{Re} \frac{1}{n} \sum_f \lambda_{\partial_i \tilde{\mathbf{Z}}}(f) e^{2\pi i f_v \delta / n_v}$ from (1) and exchanging the sums,

$$\sum_{\delta} \mu(\delta) \tilde{z}_i(\delta) = \text{Re} \frac{1}{n} \sum_f \lambda_{\partial_i \tilde{\mathbf{Z}}}(f) \hat{\mu}(f_v), \quad \hat{\mu}(f_v) := \sum_{\delta} \mu(\delta) e^{2\pi i f_v \delta / n_v} = [\text{DFT}_{n_v}(\mu)](f_v) \in \mathbb{R}, \quad (11)$$

where $\hat{\mu}$ is real (and the sign of the exponent irrelevant) because μ is even. It remains to write $\lambda_{\partial_i \bar{Z}}$ in the form of the lemma. With $\bar{Z} = \sigma_c^2 \mathbf{R}_c (\mathbf{W}^\top \mathbf{A}^{-1} \mathbf{W}) \mathbf{R}_c$, the product rule (paper's S.31–S.34) gives in the Fourier domain

$$\lambda_{\partial_i (\mathbf{W}^\top \mathbf{A}^{-1} \mathbf{W})} = \underbrace{\bar{\varepsilon}_i \lambda_A^{-1} \lambda_W}_{\mathcal{E}_i^\top \mathbf{A}^{-1} \mathbf{W}} + \underbrace{\varepsilon_i \lambda_A^{-1} \overline{\lambda_W}}_{\mathbf{W}^\top \mathbf{A}^{-1} \mathcal{E}_i} - \underbrace{|\lambda_W|^2 \lambda_A^{-2} \lambda_{\partial_i A}}_{\mathbf{W}^\top \mathbf{A}^{-1} (\partial_i A) \mathbf{A}^{-1} \mathbf{W}} = \varepsilon_i (\bar{\mathbf{G}}_2 + \mathbf{G}_3) + \overline{\varepsilon_i (\bar{\mathbf{G}}_2 + \mathbf{G}_3)}, \quad (12)$$

$$\mathbf{G}_2 := \lambda_A^{-1} \lambda_W, \quad \mathbf{G}_3 := -\sigma_c^2 |\lambda_W|^2 \lambda_A^{-2} \mathbf{G}_1,$$

so that, with $\mathbf{Q} := \sigma_c^2 \lambda_{R_c}^2 (\bar{\mathbf{G}}_2 + \mathbf{G}_3)$ (all $\mathcal{O}(n)$, independent of i),

$$\lambda_{\partial_i \bar{Z}} = \varepsilon_i \mathbf{Q} + \overline{\varepsilon_i \mathbf{Q}}. \quad (13)$$

Substituting (13) into (11) and applying the lemma to $\mathbf{G} = \hat{\mu} \odot \mathbf{Q}$ (with $\hat{\mu}$ broadcast along f_h ; real, so it can be pulled through the conjugate):

$$\text{tr}(\mathbf{Y}^{-1} \partial_i \mathbf{Y}) = -\frac{2}{n} \text{Re}[\text{DFT}_{n_v}(\hat{\mu} \odot q)](r_i), \quad q(f_v) = \sum_{f_h} \mathbf{Q}(f_v, f_h). \quad (14)$$

Cost for all k coordinates: $\mathcal{O}(m^3)$ for $\mathbf{M}$ (two $m \times m$ products after one $m \times m$ inverse), $\mathcal{O}(m^2)$ for μ , $\mathcal{O}(n)$ for $\mathbf{Q}$ and q , and two FFTs of length n_v . Compare the shipped code, which for each i forms $\lambda_{\partial_i \bar{Z}}$ on the full lattice, inverse-transforms it, extracts the $m \times m$ block, sandwiches it with $\mathbf{L}$ and multiplies by $\mathbf{Y}^{-1}$: $\mathcal{O}(k(n \log n + m^3))$.

3.4 Gradient of the sum of squares

Let $\text{SS}(\omega) = \bar{\mathbf{d}}^\top \Sigma_{d|\omega}^{-1} \bar{\mathbf{d}}$ with both $\bar{\mathbf{d}}$ and $\Sigma_{d|\omega}$ depending on ω . With $\mathbf{v} = \Sigma_{d|\omega}^{-1} \bar{\mathbf{d}}$ from (6) and $\partial_i \Sigma_{d|\omega}^{-1} = -\Sigma_{d|\omega}^{-1} (\partial_i \Sigma_{d|\omega}) \Sigma_{d|\omega}^{-1}$ (paper's eqs. 67–68),

$$\frac{\partial \text{SS}}{\partial w_i} = -\mathbf{v}^\top (\partial_i \Sigma_{d|\omega}) \mathbf{v} - 2 \mathbf{v}^\top \mathcal{E}_i \boldsymbol{\mu}_c. \quad (15)$$

Second term. $\mathcal{E}_i \boldsymbol{\mu}_c$ is linear in $\mathbf{e}_i$: writing $\mathbf{W} \boldsymbol{\mu}_c = \mathbf{\Gamma} \mathbf{w}^*$ (paper's eq. 10, $\mathbf{\Gamma}$ the image-convolution matrix built from $\boldsymbol{\mu}_c$), $\mathcal{E}_i \boldsymbol{\mu}_c = \mathbf{\Gamma} \mathbf{e}_i$, hence $-2 \mathbf{v}^\top \mathcal{E}_i \boldsymbol{\mu}_c = -2[\mathbf{\Gamma}^\top \mathbf{v}]_i$. One product $\mathbf{\Gamma}^\top \mathbf{v}$ — n_h circulant blocks applied by FFT, $\mathcal{O}(n \log n_v)$, or the precomputed dense $k \times n$ slice that the code keeps — gives all k entries. (The code already does this; it only had the wrong $\mathbf{v}$.)

First term. With $\tilde{\Sigma}_c = \sigma_c^2 (\mathbf{R}_{c,h} \otimes \mathbf{R}_{c,v} - \mathbf{R}_{c,h}^* \otimes \mathbf{R}_{c,v}^*)$ (paper's S.25), $\mathbf{W} = \mathbf{I} \otimes \mathbf{W}_0$ and $\mathcal{E}_i = \mathbf{I} \otimes \mathbf{E}_i$, the mixed-product property gives

$$\partial_i \Sigma_{d|\omega} = \mathcal{E}_i \tilde{\Sigma}_c \mathbf{W}^\top + \mathbf{W} \tilde{\Sigma}_c \mathcal{E}_i^\top = \sigma_c^2 \left[\mathbf{R}_{c,h} \otimes (\mathbf{D}_i + \mathbf{D}_i^\top) - \mathbf{R}_{c,h}^* \otimes (\mathbf{D}_i^* + \mathbf{D}_i^{*\top}) \right], \quad (16)$$

with $\mathbf{D}_i := \mathbf{E}_i \mathbf{R}_{c,v} \mathbf{W}_0^\top$ and $\mathbf{D}_i^* := \mathbf{E}_i \mathbf{R}_{c,v}^* \mathbf{W}_0^\top$, which is the paper's eq. (71). For any $\mathbf{H} \in \mathbb{R}^{n_h \times n_h}$, $\mathbf{D} \in \mathbb{R}^{n_v \times n_v}$ and $\mathbf{V} = \text{vec}^{-1}(\mathbf{v})$, the Kronecker mat-vec identity $(\mathbf{H} \otimes \mathbf{D}) \text{vec}(\mathbf{V}) = \text{vec}(\mathbf{D} \mathbf{V} \mathbf{H}^\top)$ gives

$$\mathbf{v}^\top (\mathbf{H} \otimes \mathbf{D}) \mathbf{v} = \text{tr}(\mathbf{V}^\top \mathbf{D} \mathbf{V} \mathbf{H}^\top) = \langle \mathbf{D}^\top, \mathbf{V} \mathbf{H}^\top \mathbf{V}^\top \rangle_F = \langle \mathbf{D}^\top, \mathbf{V} \mathbf{H} \mathbf{V}^\top \rangle_F \text{ for symmetric } \mathbf{H}. \quad (17)$$

This is the adjoint step: instead of applying $\mathbf{H} \otimes \mathbf{D}_i$ to $\mathbf{v}$ for every i (the code's k FFT-convolutions and its (k, n_v, n_v) tensor), form the *single* $n_v \times n_v$ matrix $\mathbf{K} := \mathbf{V} \mathbf{R}_{c,h} \mathbf{V}^\top$ (and $\mathbf{K}^* := \mathbf{V} \mathbf{R}_{c,h}^* \mathbf{V}^\top$), cost $\mathcal{O}(n_v n_h^2 + n_v^2 n_h)$, and read off the k inner products. Both $\mathbf{K}$ and $\mathbf{K}^*$ are symmetric, so $\langle \mathbf{D}_i + \mathbf{D}_i^\top, \mathbf{K} \rangle_F = 2 \langle \mathbf{D}_i, \mathbf{K} \rangle_F$.

Unconstrained part. $\mathbf{R}_{c,v} \mathbf{W}_0^\top$ is circulant with base $\mathbf{b} = \text{IDFT}_{n_v}(\lambda_{R_{c,v}} \odot \overline{\lambda_{W_0}})$, so $\mathbf{D}_i = \mathbf{E}_i \text{circ}(\mathbf{b})$ has base $b_{(\cdot - r_i)}$ and

$$\langle \mathbf{D}_i, \mathbf{K} \rangle_F = \sum_{p,q} b((q - p - r_i) \bmod n_v) K_{pq} = \sum_{\delta} b(\delta - r_i) \kappa(\delta) = \sum_u b(u) \kappa(u + r_i) = (\kappa \star \mathbf{b})(r_i), \quad (18)$$

where $\kappa(\delta) := \sum_p K_{p, (p+\delta) \bmod n_v}$ are the cyclic diagonal sums of $\mathbf{K}$; this is a cyclic cross-correlation ($\mathcal{O}(n_v^2)$ for κ) with $\mathbf{b}$: $(\kappa \star \mathbf{b}) = \text{IDFT}_{n_v}(\text{DFT}(\kappa) \odot \overline{\text{DFT}(\mathbf{b})})$, one FFT for all k shifts.

Constrained (well) part. $\mathbf{X} := \mathbf{R}_{c,v}^* \mathbf{W}_0^\top$ is a general $n_v \times n_v$ matrix ($\mathbf{R}_{c,v}^*$ is not circulant), and $\mathbf{D}_i^* = \mathbf{E}_i \mathbf{X}$ is $\mathbf{X}$ with its rows cyclically shifted by r_i , so

$$\langle \mathbf{D}_i^*, \mathbf{K}^* \rangle_F = \sum_{p,q} X_{(p+r_i) \bmod n_v, q} K_{pq}^* = \sum_q (\mathbf{K}_{:,q}^* \star \mathbf{X}_{:,q})(r_i) = \text{IDFT}_{n_v} \left(\sum_q \text{DFT}(\mathbf{X}_{:,q}) \odot \overline{\text{DFT}(\mathbf{K}_{:,q}^*)} \right)(r_i), \quad (19)$$

i.e. column-wise FFTs of two $n_v \times n_v$ matrices, $\mathcal{O}(n_v^2 \log n_v)$, again for all k at once. Collecting,

$$\boxed{-\mathbf{v}^\top (\partial_i \Sigma_{d|\omega}) \mathbf{v} = -2\sigma_c^2 \left[(\kappa \star \mathbf{b})(r_i) - \sum_q (\mathbf{K}_{:,q}^* \star \mathbf{X}_{:,q})(r_i) \right]}. \quad (20)$$

3.5 Assembling the gradient and its cost

For the free coordinates, with $\mathbf{R}_\omega$ the constrained blur prior correlation (paper’s eq. 16),

$$\frac{\partial U}{\partial w_i} = \frac{[\mathbf{R}_\omega^{-1}(\omega - \mu_\omega)]_i}{\sigma_w^2} + \frac{1}{2} \left[\underbrace{\text{tr}(\mathbf{A}^{-1} \partial_i \mathbf{A})}_{(8)} + \underbrace{\text{tr}(\mathbf{Y}^{-1} \partial_i \mathbf{Y})}_{(14)} \right] + \frac{1}{2} \left[\underbrace{-\mathbf{v}^\top (\partial_i \Sigma_{d|\omega}) \mathbf{v}}_{(20)} - 2[\mathbf{\Gamma}^\top \mathbf{v}]_i \right]. \quad (21)$$

Every bracket is now a fixed vector indexed by r_i , produced by the following work, none of which scales with k :

quantity	operations	cost
$\lambda_A, \lambda_A^{-1}, \lambda_B, \mathbf{G}_1, \mathbf{G}_2, \mathbf{G}_3, \mathbf{Q}$	element-wise on $n_v \times n_h$ arrays	$\mathcal{O}(n)$
$\bar{\mathbf{d}}, \mathbf{A}^{-1} \bar{\mathbf{d}}, \mathbf{B}^\top \bar{\mathbf{d}}, \mathbf{B} \mathbf{A}_c^\top(\cdot)$	4 BCCB mat-vecs (FFT2 pairs)	$\mathcal{O}(n \log n)$
$\mathbf{Y}, \mathbf{S}, \mathbf{S}^{-1} \mathbf{q}, \mathbf{Y}^{-1}, \mathbf{M} = \mathbf{L} \mathbf{Y}^{-1} \mathbf{L}^\top$	dense $m \times m$	$\mathcal{O}(m^3)$
μ (lag histogram of $\mathbf{M}$), $\hat{\mu}$	scatter-add, FFT	$\mathcal{O}(m^2 + n_v \log n_v)$
h_1, q and their FFTs	row sums, 2 FFTs	$\mathcal{O}(n + n_v \log n_v)$
$\mathbf{K} = \mathbf{V} \mathbf{R}_{c,h} \mathbf{V}^\top, \mathbf{K}^*, \kappa$	dense	$\mathcal{O}(n_v n_h^2 + n_v^2 n_h)$
$\mathbf{X} = \mathbf{R}_{c,v}^* \mathbf{W}_0^\top$, column FFTs of $\mathbf{X}, \mathbf{K}^*$	dense, FFT	$\mathcal{O}(n_v^3 + n_v^2 \log n_v)$
$\mathbf{\Gamma}^\top \mathbf{v}$	block-circulant mat-vec (or $k \times n$ dense)	$\mathcal{O}(n \log n_v)$ (or $\mathcal{O}(kn)$)

Total $\mathcal{O}(n \log n + m^3 + n_v^2 n_h + n_v^3)$ per gradient, versus $\mathcal{O}(k(n \log n + m^3 + n_v^2 n_h))$ for the per-coordinate scheme; at the paper’s size ($k = 126$) the measured ratio is $82\times$ (Section 2). Memory drops from ~ 1 GB of k -indexed temporaries to a few $n_v \times n_v$ and $n_v \times n_h$ arrays. In automatic-differentiation terms: the shipped code is forward mode (k Jacobian-vector products, one per basis direction $\mathbf{e}_i$), while (8), (14), (20) are a single reverse-mode sweep in which $\mathbf{A}^{-1}, \mathbf{M}, \mathbf{K}, \mathbf{K}^*$ play the role of the adjoint (cotangent) variables and the final FFT-at- r_i is the transpose of the linear map $\mathbf{w}^* \mapsto \mathbf{W}_0$.

4 The momentum is not preconditioned (bug)

`collapsedhmc/initialize.py::_create_momentum_object:`

```
precondition = sampler.mcmc_config.get('precondition', None) # key lives in mcmc_config['collapsed_hmc'] -> always None
if precondition == 'prior':
    sigma2p = 1 / sampler.theta['sigma2w'][:, 0]
    mass_correlation_matrix = _w.wavelet_constraints['inv_R_wu_star'] # a scipy.sparse csr_matrix
else:
    sigma2p = sampler.mcmc_config['collapsed_hmc']['sigma2p']
    mass_correlation_matrix = np.eye(dim_p)
...
M = Covariance(lattice=lattice_p, sigma2=sigma2p, R=mass_correlation_matrix) # calls linalg.inv(R): fails on csr
```

`_initialize_collapsed_hmc` stores the option under `mcmc_config['collapsed_hmc']['precondition']`, and `kinetic/grad_kinetic` read it from there, so the sampler is internally consistent (valid HMC) but with $\mathbf{M} = \sigma_w^{-2} \mathbf{I}$: only the scalar part of Σ_ω^{-1} , not the correlation structure $\mathbf{R}_\omega^{-1}$. Fixing the key alone raises `ValueError: Sparse arrays/matrices are not supported inside Covariance`, so the intended path was never exercised. Its impact in my experiments was modest ($\text{ESS } 128 \rightarrow 150$ on $n = 432$) — the gradient bug dominates — but the paper’s description of the algorithm does not match what ran. A more useful choice for this likelihood-dominated posterior ($\text{SNR} \approx 116 \text{ dB}$) would be an estimate of the posterior precision of ω ($k \times k$, cheap from a pilot run), which also lets a fixed $L = 10$ traverse the posterior.

5 End-to-end ESS/s: what the fixes buy

Same simulated set-up as the notebook (full well column, endpoint-constrained blur, hyper-parameters $\alpha_w = 2.01$, $\beta_w = 10$, $\alpha_c = 2.00001$, $\beta_c = 1/500$, $\alpha_\zeta = 3$, $\beta_\zeta = 0.1$, $\rho = 1.5$), chains initialised at the truth, $L = 10$, ε adapted multiplicatively in batches of 25 to a 70% acceptance target, first 20% of the chain discarded. ESS uses the repository’s `estimate_effective_sample_size` (Geyer initial monotone sequence) on the free blur coordinates. Chains with identical seeds are identical across the “fixed” and “adjoint” rows, a second check that the adjoint gradient is the same function.

36 × 12 lattice, $k = 12$, $m = 24$ ($n = 432$); Gibbs 3000 iterations, HMC 1500

variant	ms/iter	accept.	ESS(ω) min	ESS(ω) med.	ESS/s (med.)	final ε
Gibbs	11.3	—	48.4 / 2400	72.3 / 2400	2.67	—
HMC as shipped (wrong gradient, identity mass)	31.0	0.71	3.4 / 1200	5.1 / 1200	0.14	0.0003
HMC, gradient fixed (identity mass)	29.4	0.71	55.3 / 1200	127.9 / 1200	3.62	0.082
HMC, gradient fixed + prior mass matrix	30.9	0.72	71.3 / 1200	149.9 / 1200	4.04	0.114
same, adjoint gradient + potential	20.1	0.72	71.3 / 1200	149.9 / 1200	6.23	0.114

90 × 24 lattice, $k = 30$, $m = 60$ ($n = 2160$); Gibbs 2000 iterations, HMC 800

variant	ms/iter	accept.	ESS(ω) min	ESS(ω) med.	ESS/s (med.)	final ε
Gibbs	19.8	—	7.5 / 1600	22.6 / 1600	0.71	—
HMC as shipped (wrong gradient, identity mass)	144.2	0.69	3.3 / 640	5.1 / 640	0.056	0.0001
HMC, gradient fixed + prior mass matrix	117.1	0.76	27.7 / 640	59.3 / 640	0.79	0.063
same, adjoint gradient + potential	39.1	0.76	27.7 / 640	59.3 / 640	2.37	0.063

With the shipped code HMC is 13–20× *worse* than Gibbs per second, reproducing the paper’s qualitative finding. Fixing the gradient alone makes each HMC draw worth 4–7 Gibbs draws; the adjoint gradient then makes the HMC iteration cheap enough that HMC wins per second by $\sim 3\times$ at $n = 2160$. The remaining per-iteration overhead of the adjoint version is independent of k , so the gap should widen at the real example ($k = 126$, where the vectorised gradient is 82× slower than the adjoint one). Absolute times are from a 2-vCPU sandbox with single-threaded BLAS; compare ratios, not seconds, with the paper’s 13 s / 0.47 s.

6 Secondary inefficiencies (both samplers)

- **subset_AB builds a dense $m \times n$ matrix** (100 circulant 480×480 blocks + `hstack`, ~ 370 MB of allocation) each time it is called, just to apply $A_c B^\top$ or $R A_c^\top$ to a vector. In `potential()` this is $A_c B^\top \bar{d} = (B^\top \bar{d})[\text{well rows}]$: one FFT mat-vec and a gather. The same call sits in the **Gibbs image update** (`RAt = subset_AB(base_R_cond, _image).T`) and is why that update costs 382 ms of the ~ 0.45 s Gibbs iteration; $R_{c|d} A_c^\top x$ is a scatter followed by an FFT mat-vec. Gibbs itself could be 5–10× faster.
- **_correct_sample computes the Kriging correction twice** (once with `inv_ARAt`, once with a Cholesky solve) and keeps the first; the second exists only for a verbose diagnostic but always runs, including an $n \times m$ mat-vec and an $m \times m$ Cholesky.
- **potential() recomputes A, Z, Y, S, v** that the gradient at the same point has just computed; two potentials per iteration cost as much as two more gradients in the adjoint version. `potential_adjoint/gradient_potential_adjoint` share a common dictionary for this.
- **kinetic/grad_kinetic re-factorise the mass matrix** (Cholesky of a $k \times k$ matrix) at every leapfrog step.
- **Traces via full products.** `np.trace(inv_Y @ dY, axis1=1, axis2=2)` forms k full $m \times m$ products to read k traces (`einsum('ab,kba->k')` is m^2 per coordinate) — moot with the adjoint version.
- **create_W mutates its input** (`w_star[np.abs(w_star) < 1e-11] = 0`), including the leapfrog position and the column of `sampler.theta['w_star']` it was sliced from. Numerically negligible, but it breaks exact reversibility and is easy to remove.
- **Memory at set-up.** Covariance allocates `np.eye(n)` (17 GB at $n = 48000$) to obtain one unit vector; each Wavelet object stores ~ 1.1 GB of derivative tensors (`bases_d_W`, `bases_d_WT`, `fft2_bases_d_W`, `fft2_bases_d_WT`) of which only `fft2_bases_d_WT` and `bases_d_W0` are read at iteration time, and the model is deep-copied several times; `_initialize_collapsed_hmc` builds another ~ 1.4 GB (`dW0_Rcv_star`, `bases_d_W`

Rc, gamma) that the gradient never reads. The real example needs well over 8 GB before the first iteration for no algorithmic reason.

- **Paper text.** The per-leapfrog complexity on p. 23 should read $\mathcal{O}(k(n \log n + m^3 + n(n_v + n_h)))$ for the algorithm as derived and implemented, or Section 4.2.2 should be replaced by the adjoint form of Section 3, which does achieve the stated bound. Section 4.2 should also state the mass matrix that was actually used ($\mathbf{M} = \sigma_w^{-2} \mathbf{I}$).

7 Recommended order of changes

1. Apply the three-line patch in Appendix A (P.copy(), the precondition key + .toarray(), and np.eye(1, n)). Re-run check_gradient.py (Appendix C); the FD error should be $\sim 10^{-7}$.
2. Swap gradient_potential_Fourier_domain/potential for the adjoint versions in cyclic/utls.py (leapfrog_collapsed_c and log_ar_hmc_collapsed_c); pass the common dictionary from the first/last gradient into the two potential evaluations.
3. Replace subset_AB in potential() and in the Gibbs image update by an FFT mat-vec plus gather/scatter, and drop the duplicated correction in _correct_sample.
4. Re-tune HMC: with a correct force, ε will be 100–300 $\times$ larger than before at the same acceptance rate; consider a $k \times k$ mass matrix from a pilot run’s posterior covariance and re-examine L . Then redo the Table 1 / Figure 5 comparison — the ESS/s conclusion is likely to reverse.

A Minimal patch (three files)

Apply from the repository root with `git apply bsbd_minimal_fixes.patch`.

```
1 diff --git a/src/classes/covariance.py b/src/classes/covariance.py
2 index a9f3c5d..4941002 100644
3 --- a/src/classes/covariance.py
4 +++ b/src/classes/covariance.py
5 @@ -136,7 +136,7 @@ class Covariance():
6
7         # If no base_R is passed, assume R=I
8     else:
9         self.base_R = np.eye(self.n)[0, :].reshape(-1, 1)
10 +         self.base_R = np.eye(1, self.n)[0, :].reshape(-1, 1) # avoid allocating an n x n identity (17 GB at n
11         ↪ =48000)
12         self.base_Q = invert_circ(self.base_R)
13         self.logdet_R = compute_log_det_circ(self.base_R)
14 diff --git a/src/mcmc/update_blur_image/collapsedhmc/cyclic/gradient_potential.py b/src/mcmc/update_blur_image/collapsedhmc/
15 ↪ cyclic/gradient_potential.py
16 index ecf2e59..586287b 100644
17 --- a/src/mcmc/update_blur_image/collapsedhmc/cyclic/gradient_potential.py
18 +++ b/src/mcmc/update_blur_image/collapsedhmc/cyclic/gradient_potential.py
19 @@ -216,7 +216,10 @@ def gradient_potential_Fourier_domain(sampler, i, w_star):
20
21     ### Compute P_R_ch_starT and R_cv_star_W0t in the time domain, used to compute the gradient of SS
22     if _c.reflectivity_constraints['constr']:
23         P = p.reshape((nv, nh), order='F')
24         # NOTE: reshape(order='F') of the (n,1) vector p returns a VIEW of p. gradient_SS_vectorized
25         # then runs np.fft.fft(P, axis=1, out=P) etc. in place, which overwrote p before it was
26         # used in `... @ p` (d_SS_1 part 1/2 and d_SS_2). Take a copy so p stays Sigma^-1 dbar.
27         P = p.reshape((nv, nh), order='F').copy()
28         R_ch_star = _c.reflectivity_constraints['Rh_star']
29         P_R_ch_starT = P @ (R_ch_star.T)
30         WOT = linalg.circulant(base_WOT.reshape(-1)).T
31 diff --git a/src/mcmc/update_blur_image/collapsedhmc/initialize.py b/src/mcmc/update_blur_image/collapsedhmc/initialize.py
32 index 7257959..18a3700 100644
33 --- a/src/mcmc/update_blur_image/collapsedhmc/initialize.py
34 +++ b/src/mcmc/update_blur_image/collapsedhmc/initialize.py
35 @@ -190,10 +190,14 @@ def _create_momentum_object(sampler):
36     dim_p = _lik.lattice.nv - _w.wavelet_constraints['nr_constraints']
37
38     # Precondition?
39     precondition = sampler.mcmc_config.get('precondition', None)
40     # NOTE: the option is stored under mcmc_config['collapsed_hmc'] (see _initialize_collapsed_hmc);
41     # reading it from the top level always gave None, i.e. an identity mass matrix.
42     precondition = sampler.mcmc_config['collapsed_hmc'].get('precondition', None)
43     if precondition == 'prior':
44         sigma2p = 1 / sampler.theta['sigma2w'][:, 0]
45         mass_correlation_matrix = _w.wavelet_constraints['inv_R_wu_star']
46         if hasattr(mass_correlation_matrix, 'toarray'):
47             mass_correlation_matrix = mass_correlation_matrix.toarray() # stored as csr; Covariance() needs dense
48         print(f"Using prior preconditioning with mass_correlation_matrix shape: {mass_correlation_matrix.shape}") if verbose
49         ↪ else None
50     else:
51         sigma2p = sampler.mcmc_config['collapsed_hmc']['sigma2p']
```

B gradient_adjoint.py

Drop-in module (path `src/mcmc/update_blur_image/collapsedhmc/cyclic/gradient_adjoint.py`) implementing Section 3. `gradient_potential_adjoint(sampler, i, w_star)` and `potential_adjoint(sampler, i, w_star)` have the same signatures and return values as the existing `gradient_potential_Fourier_domain` and `potential`; both accept an optional `common=` argument to share intermediates (`gradient_potential_adjoint(..., return_common=True)` returns it). Variable names follow the derivation: $G_1, G_2, G_3, Q, h_1, q; M, M_{\text{lag}}, hM = M, \mu, \hat{\mu}; K, K_{\text{diag}}, b = K, \kappa, b; X, K_s = X, K^*; r_{\text{idx}} = \{r_i\}$.

```

1  """Adjoint (reverse-mode) evaluation of the collapsed-HMC potential gradient
2  and a leaner potential, cyclic case.
3
4  The existing gradient_potential_Fourier_domain() evaluates, for *each* of the
5  k free blur coordinates, a directional derivative in the Fourier domain: it
6  materialises (k, nv, nh) complex tensors, k batched (m x m) sandwiches
7  L' dZ_i L, k batched (m x m) products inv_Y @ dY_i, k IFFT2s and a
8  (k, nv, nv) tensor for the constrained-image term. Its cost is therefore
9  O(k * (n log n + m^3 + nv^2 nh)) per leapfrog step and it moves ~1 GB of
10 temporaries per call at the paper's problem size.
11
12 Every one of those k quantities is linear in the derivative direction
13 eps_i(f_v) = exp(-2 pi i f_v r_i / nv) (the DFT of the shifted unit vector
14 that generates dW0/dw_i), so all k coordinates can be obtained from ONE
15 n-sized reduction followed by ONE length-nv FFT evaluated at the k shifts
16 r_i. This is what this module does. Cost per gradient:
17 O(n log n + m^3 + nv^2 nh) with no k-fold tensors at all.
18
19 Numerically identical (to ~1e-12 relative) to the fixed vectorised gradient.
20 """
21 import math
22 import numpy as np
23 import scipy.fft
24 from scipy import linalg
25
26 from src.utils.model_utils import create_W, embed_wu
27 from src.utils.efficient_algebra_utils import transpose_base_circ, multiply_matrix_vector_circ
28 from src.utils.math_utils import compute_log_det_circ, compute_log_det
29 from src.mcmc.update_blur_image.collapsedhmc.cyclic.efficient_utils import (
30     compute_eigenvalues_A_invA_Z_B, project_down_with_Ac, project_up_with_Ac)
31
32
33 def _common(sampler, i, w_star):
34     """Quantities shared by potential and gradient (all O(n log n + m^3))."""
35     _lik = sampler.par_objs['d']; _w = sampler.par_objs['w']; _c = sampler.par_objs['c']
36     nv, nh = _lik.lattice.nv, _lik.lattice.nh
37     n = nv * nh
38     sigma2c = sampler.theta['sigma2c'][0, i + 1].item()
39     d = sampler.theta['d_star'][:, i + 1].reshape((-1, 1))
40     constr_c = _c.reflectivity_constraints['constr']
41
42     W0, W, base_W0, base_W, base_W0T, base_WT = create_W(_lik.lattice, w_star)
43     ev_W0 = scipy.fft.fft(base_W0.flatten())
44     ev_W = np.repeat(ev_W0[:, None], nh, axis=1)
45     ev_WT = np.conj(ev_W)
46     eigs, base_Z = compute_eigenvalues_A_invA_Z_B(sampler, i, ev_W, ev_WT)
47     base_Z = base_Z.real
48
49     out = dict(nv=nv, nh=nh, n=n, sigma2c=sigma2c, d=d, constr_c=constr_c, eigs=eigs,
50               base_Z=base_Z, ev_W0=ev_W0, ev_W=ev_W, ev_WT=ev_WT, base_W=base_W, base_W0T=base_W0T)
51
52     # collapsed-likelihood mean and centred data (mu_c is the constrained prior mean)
53     mean_c = _c.reflectivity_constraints['mean_c_star'] if constr_c else _c.mean
54     mean_c_hat = scipy.fft.ifft2(mean_c.reshape((nv, nh), order='F'))
55     mean_lik = scipy.fft.fft2(ev_W * mean_c_hat).real.reshape((n, 1), order='F')
56     dbar = d - mean_lik
57     D = dbar.reshape((nv, nh), order='F')
58     Dhat = scipy.fft.ifft2(D)
59     out['dbar'] = dbar
60     # r = A^-1 dbar
61     r = scipy.fft.fft2(eigs['eigenvalues_inv_A'] * Dhat).real.reshape((n, 1), order='F')
62     out['r'] = r
63
64     if constr_c:
65         rows = _c.lattice.well_positions['rows_cyclic']
66         L = _c.reflectivity_constraints['L']
67         m = _c.reflectivity_constraints['nr_constraints']
68         # A_c Z A_c' and S = A_c (Sigma_c - Z) A_c' from the diagonal block base
69         bz = base_Z[:, 0]
70         AZAt = bz[(rows[None, :] - rows[:, None]) % nv] # C[p,q] = base[(q-p) mod nv]

```

```

71     AZAt = 0.5 * (AZAt + AZAt.T)
72     bsc = sigma2c * _c.Sigma.base_R[:, 0]
73     S = (bsc - bz)[(rows[None, :] - rows[:, None]) % nv]
74     S = 0.5 * (S + S.T)
75     Y = np.eye(m) - L.T @ AZAt @ L / sigma2c
76     Y = 0.5 * (Y + Y.T)
77     cS = linalg.cho_factor(S, lower=True)
78     # A_c B' dbar via one FFT mat-vec + gather (no dense m x n matrix)
79     Btd = scipy.fft.fft2(eigs['eigenvalues_BT'] * Dhat).real.reshape((n, 1), order='F')
80     q = project_down_with_Ac(_c, Btd) # m x 1
81     Sinv_q = linalg.cho_solve(cS, q)
82     t = project_up_with_Ac(_c, Sinv_q)
83     s = scipy.fft.fft2(eigs['eigenvalues_B'] * scipy.fft.ifft2(t.reshape((nv, nh), order='F'))).real.reshape((n, 1), order=
    ↪ 'F')
84     out.update(rows=rows, L=L, m=m, Y=Y, S=S, cS=cS, q=q, Sinv_q=Sinv_q, s=s, p=r + s)
85 else:
86     out.update(p=r)
87 return out
88
89
90 def potential_adjoint(sampler, i, w_star, common=None):
91     """Same value as potential(), without building any dense m x n matrix."""
92     _w = sampler.par_objs['w']
93     c = common if common is not None else _common(sampler, i, w_star)
94     nv, n, sigma2c = c['nv'], c['n'], c['sigma2c']
95     sigma2w = sampler.theta['sigma2w'][0, i + 1].item()
96
97     # prior
98     nr = _w.wavelet_constraints['nr_constraints']
99     unc = _w.wavelet_constraints['unconstrained_indices']
100    wc = (w_star[unc, :] - _w.wavelet_constraints['mean_w_star'])[unc, :].reshape(-1, 1)
101    z = linalg.solve_triangular(_w.wavelet_constraints['chol_R_wu_star'], wc, lower=True)
102    U_w = 0.5 * (nv - nr) * math.log(2 * math.pi) + 0.5 * (nv - nr) * math.log(sigma2w) \
103        + 0.5 * _w.wavelet_constraints['logdet_R_wu_star'] + 0.5 * float((z.T @ z)[0, 0]) / sigma2w
104
105    # likelihood: log|A| + log|Y| and dbar' A^-1 dbar + q' S^-1 q
106    logdet_A = float(np.sum(np.log(c['eigs']['eigenvalues_A'].real)))
107    ss = float((c["dbar"].T @ c["r"])[0, 0])
108    logdet_Y = 0.0
109    if c['constr_c']:
110        logdet_Y = compute_log_det(c['Y'])
111        ss += float((c["q"].T @ c["Sinv_q"])[0, 0])
112    U_d = 0.5 * n * math.log(2 * math.pi) + 0.5 * (logdet_A + logdet_Y) + 0.5 * ss
113    return U_d + U_w
114
115
116 def gradient_potential_adjoint(sampler, i, w_star, common=None, return_common=False):
117     """Gradient of the potential w.r.t. w* (nv x 1, zero on constrained coords)."""
118     _w = sampler.par_objs['w']; _c = sampler.par_objs['c']
119     c = common if common is not None else _common(sampler, i, w_star)
120     nv, nh, n, sigma2c = c['nv'], c['nh'], c['n'], c['sigma2c']
121     eigs = c['eigs']
122     unc = _w.wavelet_constraints['unconstrained_indices']
123     # r_i: position of the single 1 in base(dw0/dw_i) (same convention as the vectorised code)
124     bases_d_W0 = _w.wavelet_derivatives['bases_d_W0'][:, unc]
125     r_idx = np.argmax(bases_d_W0, axis=0)
126
127     # ----- prior -----
128     sigma2w = sampler.theta['sigma2w'][0, i + 1].item()
129     wc = (w_star[unc, :] - _w.wavelet_constraints['mean_w_star'])[unc, :].reshape(-1, 1)
130     chol = _w.wavelet_constraints['chol_R_wu_star']
131     g_prior = linalg.solve_triangular(chol.T, linalg.solve_triangular(chol, wc, lower=True), lower=False) / sigma2w
132
133     # ----- log-determinant -----
134     invA = eigs['eigenvalues_inv_A'].real # A is symmetric BCCB -> real eigenvalues
135     lam_Rc = _c.Sigma.fft_base_R # (nv, nh) eigenvalues of R_c (real)
136     G1 = lam_Rc * c['ev_WT'] # dA_i = sigma2c (eps_i G1 + conj)
137     h1 = (invA * G1).sum(axis=1) # length nv
138     tr_invA_dA = 2.0 * sigma2c * scipy.fft.fft(h1).real[r_idx]
139
140     tr_invY_dY = np.zeros(len(unc))
141     if c['constr_c']:
142         L, Y, rows, m = c['L'], c['Y'], c['rows'], c['m']
143         inv_Y = linalg.inv(Y); inv_Y = 0.5 * (inv_Y + inv_Y.T)
144         M = L @ inv_Y @ L.T # m x m, once
145         Mlag = np.zeros(nv)
146         np.add.at(Mlag, (rows[None, :] - rows[:, None]) % nv, M) # lag histogram of M (symmetric in lag)
147         hM = scipy.fft.fft(Mlag).real # real because Mlag[d] = Mlag[-d]
148         G2 = invA * c['ev_W']
149         G3 = -sigma2c * (np.abs(c['ev_W']) ** 2) * invA ** 2 * G1
150         Q = sigma2c * lam_Rc ** 2 * (np.conj(G2) + G3) # eig(dZ_i) = eps_i Q + conj(eps_i Q)
151         qv = Q.sum(axis=1)
152         tr_invY_dY = -(2.0 / n) * scipy.fft.fft(hM * qv).real[r_idx]

```

```

153 g_logdet = tr_invA_dA + tr_invY_dY
154
155 # ----- sum of squares -----
156 p = c['p'].real
157 P = p.reshape((nv, nh), order='F')
158 # part 1:  $p'(R_{ch} \otimes D1_i)p = 2 \langle Dh_i, K \rangle$ ,  $K = P R_{ch} P'$ ,  $Dh_i = circ(roll(base(R_{cv} WO'), r_i))$ 
159 Rch = _c.Sigma.Rh
160 K = P @ Rch @ P.T
161 Kdiag = np.array([np.trace(K, offset=dlt) + (np.trace(K, offset=dlt - nv) if dlt > 0 else 0.0) for dlt in range(nv)])
162 b = scipy.fft.ifft(_c.Sigma.fft_base_Rv.ravel() * np.conj(c['ev_WO'])).real # base of  $R_{cv} WO'$ 
163 part1 = 2.0 * scipy.fft.ifft(np.conj(scipy.fft.fft(b)) * scipy.fft.fft(Kdiag)).real[r_idx]
164 part2 = np.zeros(len(unc))
165 if c['constr_c']:
166     Rh_star = _c.reflectivity_constraints['Rh_star']; Rv_star = _c.reflectivity_constraints['Rv_star']
167     WOT = linalg.circulant(c['base_WOT'].reshape(-1)).T
168     X = Rv_star @ WOT #  $nv \times nv$ 
169     Ks = P @ Rh_star @ P.T
170     cc = scipy.fft.ifft((scipy.fft.fft(X, axis=0) * np.conj(scipy.fft.fft(Ks, axis=0))).sum(axis=1)).real
171     part2 = 2.0 * cc[r_idx]
172 d_SS_1 = -sigma2c * (part1 - part2)
173 gamma_u = sampler.aux_derivatives['gamma_matrix_unconstrained_indexes']
174 d_SS_2 = -2.0 * (gamma_u @ p).ravel()
175 g_ss = d_SS_1 + d_SS_2
176
177 g_u = g_prior.ravel() + 0.5 * g_logdet + 0.5 * g_ss
178 grad = embed_wu(g_u.reshape(-1, 1), _w)
179 return (grad, c) if return_common else grad

```

C check_gradient.py (verification and timing harness)

Run from the repository root, e.g. `python check_gradient.py 24 6 12 12 6` ($n = 432$, a few seconds) or `python check_gradient.py 330 50 126 150 50 2` (paper size; needs several GB of RAM because of the set-up allocations noted in Section 6). Arguments: observed rows, observed columns, blur length k , vertical margin, horizontal margin, [timing repetitions]. It builds the notebook's simulated model, compares the repo gradient, the adjoint gradient and central finite differences of `potential()`, and times each. Note the shim for `scipy.linalg.kron`, which no longer exists in the SciPy version the repository pins.

```
1  """Standalone check of the collapsed-HMC gradient (cyclic case).
2
3  Compares (1) the repo gradient gradient_potential_Fourier_domain, (2) the adjoint
4  gradient in gradient_adjoint.py and (3) central finite differences of potential().
5  Also times each. Run from the repo root:
6
7      python check_gradient.py NV_OBS NH_OBS K MV MH [reps]
8
9  e.g.  python check_gradient.py 24 6 12 12 6      (n=432, quick)
10       python check_gradient.py 330 50 126 150 50 2  (paper size; needs ~6 GB RAM)
11 """
12 import os, sys, time, random, copy, math, contextlib, io
13 import numpy as np
14 from scipy import linalg
15
16 root_dir = os.path.dirname(os.path.abspath(__file__))
17 sys.path.insert(0, root_dir)
18 import scipy.linalg
19 if not hasattr(scipy.linalg, 'kron'):
20     scipy.linalg.kron = np.kron
21
22 from src.classes.lattice import Lattice
23 from src.classes.model import ConvolutionalModel
24 from src.mcmc.mcmc import MCMC
25 from src.mcmc.update_blur_image.collapsedhmc.cyclic.potential import potential
26 from src.mcmc.update_blur_image.collapsedhmc.cyclic import gradient_potential as gpmod
27 from src.utils.model_utils import create_W, subset_AB, subset_ABA, embed_wu
28 from src.mcmc.update_blur_image.collapsedhmc.cyclic.efficient_utils import compute_bases
29 from src.utils.efficient_algebra_utils import transpose_base_circ, multiply_matrix_vector_circ
30 from src.utils.math_utils import compute_log_det_circ, compute_log_det
31
32 nv_obs, nh_obs, k, mv, mh = [int(a) for a in sys.argv[1:6]]
33 reps = int(sys.argv[6]) if len(sys.argv) > 6 else 3
34 nwell = None
35 seed = 20
36 random.seed(seed); np.random.seed(seed)
37 alpha_w, beta_w = 2.01, 10
38 alpha_c, beta_c = 2.00001, 1 / 500
39 alpha_zeta, beta_zeta = 3, 0.1
40 zeta_init = 0.05
41 rho_w = rho_v = rho_h = 1.5
42
43 def build_model(lattice, observed_b=None, image_b=None):
44     model = ConvolutionalModel(lattice)
45     model.setup_wavelet_variance(alpha=alpha_w, beta=beta_w); model.initialize_wavelet_variance(init=None)
46     model.setup_reflectivity_variance(alpha=alpha_c, beta=beta_c); model.initialize_reflectivity_variance(init=None)
47     model.setup_inverse_snr(alpha=alpha_zeta, beta=beta_zeta); model.initialize_inverse_snr(init=zeta_init)
48     model.setup_wavelet_prior(rho=rho_w, constr=True); model.initialize_wavelet()
49     model.setup_reflectivity_prior(rho_v=rho_v, rho_h=rho_h, b=image_b); model.initialize_reflectivity()
50     model.setup_seismic_model(rho_v=rho_v, rho_h=rho_h, b=observed_b); model.initialize_seismic()
51     if os.environ.get('BSBD_SLIM', '0') == '1': # drop precomputed arrays that are never used at iteration time (memory only)
52         wd = model.model['w'].wavelet_derivatives
53         for key in ['fft2_bases_d_W', 'bases_d_WT', 'bases_d_W', 'd_W0', 'd_W']:
54             wd[key] = None
55     return model
56
57 def setup(nv_obs, nh_obs, k, mv, mh, nwell):
58     with contextlib.redirect_stdout(io.StringIO()):
59         lattice = Lattice(nv_obs, nh_obs, k, mv=mv, mh=mh, topology='C')
60         lattice.create_ava_positions(verbose=False); lattice.create_wavelet_positions(verbose=False)
61         data_true = build_model(lattice)
62         if nwell == 0:
63             image_b = None
64         else:
65             if nwell is None:
66                 rows_obs = np.arange(lattice.nv_ava, dtype=int)
67             else:
68                 mid = lattice.nv_ava // 2; start = mid - nwell // 2
69                 rows_obs = np.arange(start, start + nwell, dtype=int)
70             lattice.create_obs_reflectivity_positions(well_column_ava=None, rows_ava=rows_obs, verbose=False)
```

```

71     well_coords_vec = lattice.well_positions['well_coords_vec']
72     image_b = data_true.theta['c_star'].reshape(-1)[well_coords_vec].reshape(-1, 1)
73     data_model = copy.deepcopy(data_true)
74     data_model.setup_reflectivity_prior(rho_v=rho_v, rho_h=rho_h, b=image_b)
75     data_model.initialize_reflectivity(init=data_true.theta['c_star'])
76     estim_root = os.path.join(root_dir, 'estimations', 'prof'); os.makedirs(estim_root, exist_ok=True)
77     estimate = ['c_star', 'w_star', 'sigma2c', 'sigma2w', 'zeta']
78     if data_model.lattice.n > data_model.lattice.n_ava: estimate.append('d_star')
79     sampler = MCMC(model=data_model, theta_init=data_true.theta, estimate=estimate,
80                   theta_true=data_model.theta, chunk_size=10, path=estim_root + '/',
81                   model_filename='gradcheck', folder='gradcheck/')
82     adapt_config = {'collapsed_hmc': {'epsilon': {'type': None}, 'L': {'type': None}}}
83     sampler.mcmc_config = {'algorithm': 'collapsed_hmc', 'N': 10, 'verbose': False, 'save_stats': False,
84                           'constr_SSD': False, 'constr_SSW': True, 'constr_SSC': False}
85     sampler._create_par_objects(False)
86     sampler.adapt_config = adapt_config
87     sampler.adapt_tracking = {m: {p: {'history': np.zeros(11), 'batch_ar': []} for p in pd} for m, pd in adapt_config.items}
88     ↪ ()
89     sampler.stats = {'mwg': {'init_time': time.time(), 'exec_time': 0, 'error': np.zeros(11)},
90                     'wc_update': {'exec_time': None, 'iter_with_chmc': np.zeros(11), 'nr_iters': np.zeros(11),
91                                   'loglik': np.zeros(11), 'log_prior_w': np.zeros(11), 'log_prior_c': np.zeros(11),
92                                   'log_posterior_dens': np.zeros(11),
93                                   'acceptance': {'log_acc_prob': np.zeros(11), 'acc_iter': np.zeros(11), 'cum_ar': np.
94                                   ↪ zeros(11)}},
95                     'log_post': {kk: np.zeros((1, 1)) for kk in ['likelihood', 'prior_reflectivity', 'prior_wavelet',
96                     ↪ 'prior_sigma2c', 'prior_sigma2w', 'prior_zeta', 'joint']}}
97     sampler._create_sample_aux_dicts(N=11)
98     sampler._initialize_sample_aux_dicts(i=0)
99     from src.mcmc.mwg.cyclic.update_ss import _update_ss
100     _update_ss(sampler, -1)
101     from src.mcmc.update_blur_image.collapsedhmc.initialize import _initialize_collapsed_hmc, _create_momentum_object
102     _initialize_collapsed_hmc(sampler, epsilon=0.008, L=10, sigma2p=1, precondition='prior', p_collapsed_hmc=1)
103     _create_momentum_object(sampler)
104     from src.mcmc.updates.state import _copy_previous_state
105     _copy_previous_state(sampler, 0)
106     return sampler, lattice
107
108 def _unused_potential_parts(sampler, i, w_star):
109     """Copy of potential() returning the pieces: (prior, logdet_A, logdet_Y, ss1, ss2)."""
110     _lik = sampler.par_objs['d']; _c = sampler.par_objs['c']; _w = sampler.par_objs['w']
111     d_star = sampler.theta['d_star'][:, i + 1].reshape((-1, 1))
112     sigma2w = sampler.theta['sigma2w'][0, i + 1].item(); sigma2c = sampler.theta['sigma2c'][0, i + 1].item()
113     unc = _w.wavelet_constraints['unconstrained_indices']
114     mean_w_star = _w.wavelet_constraints['mean_w_star']
115     wc = (w_star[unc, :] - mean_w_star[unc, :]).reshape(-1, 1)
116     chol = _w.wavelet_constraints['chol_R_wu_star']
117     z = linalg.solve_triangular(chol, wc, lower=True)
118     prior = 0.5 * (z.T @ z)[0, 0] / sigma2w
119     W0, W, base_W0, base_W, base_W0T, base_WT = create_W(_lik.lattice, w_star.copy())
120     base_A, base_inv_A, base_Z, base_B = compute_bases(sampler, i, base_W, base_WT)
121     logdet_A = compute_log_det_circ(base_A)
122     constr_c = _c.reflectivity_constraints['constr']
123     if constr_c:
124         n_w = _c.reflectivity_constraints['nr_constraints']; rows_cyclic = _c.lattice.well_positions['rows_cyclic']
125         mean_c = _c.reflectivity_constraints['mean_c_star']; L = _c.reflectivity_constraints['L']
126         AZAt = linalg.circulant(base_Z[:, 0]).T[rows_cyclic, :][:, rows_cyclic]
127         Y = np.eye(n_w) - L.T @ AZAt @ L / sigma2c
128         logdet_Y = compute_log_det(Y)
129     else:
130         mean_c = _c.mean; logdet_Y = 0.0
131     mean_lik = multiply_matrix_vector_circ(base_W, mean_c)
132     dc = d_star - mean_lik
133     ss1 = (dc.T @ multiply_matrix_vector_circ(base_inv_A, dc))[0, 0]
134     ss2 = 0.0
135     if constr_c:
136         ABt = subset_AB(transpose_base_circ(base_B), _c)
137         S = subset_ABA(sigma2c * _c.Sigma.base_R - base_Z, _c)
138         ABt_d = ABt @ dc
139         ss2 = (ABt_d.T @ linalg.solve(S, ABt_d))[0, 0]
140     return dict(prior=prior, logdet_A=float(logdet_A), logdet_Y=float(logdet_Y), ss1=float(ss1), ss2=float(ss2))
141
142 # -----
143 from src.mcmc.update_blur_image.collapsedhmc.cyclic.gradient_potential import gradient_potential_Fourier_domain
144 from src.mcmc.update_blur_image.collapsedhmc.cyclic.potential import potential
145 from src.mcmc.update_blur_image.collapsedhmc.cyclic.gradient_adjoint import gradient_potential_adjoint, potential_adjoint,
146 ↪ _common
147
148 sampler, lattice = setup(nv_obs, nh_obs, k, mv, mh, nwell)
149 nv, nh, n = lattice.nv, lattice.nh, lattice.n
150 _w = sampler.par_objs['w']; _c = sampler.par_objs['c']
151 unc = _w.wavelet_constraints['unconstrained_indices']; m = _c.reflectivity_constraints['nr_constraints']
152 print(f"nv={nv} nh={nh} n={n} k={lattice.k} k_free={len(unc)} m={m}")

```

```

150 i = 0
151 w0 = sampler.theta['w_star'][:, i + 1].reshape((-1, 1)).copy()
152 w0[unc, 0] += 0.3 * np.std(w0[unc, 0]) * np.random.randn(len(unc))
153
154 U_ref = potential(sampler, i, w0.copy()); U_adj = potential_adjoint(sampler, i, w0.copy())
155 print(f"potential: repo {U_ref:.10g} adjoint {U_adj:.10g} rel diff {abs(U_ref-U_adj)/abs(U_ref):.2e}")
156 g_ref = gradient_potential_Fourier_domain(sampler, i, w0.copy())[unc, 0]
157 g_adj = gradient_potential_adjoint(sampler, i, w0.copy())[unc, 0]
158 print(f"gradient: max |adjoint - repo(fixed)| / max|repo| = {np.abs(g_adj - g_ref).max() / np.abs(g_ref).max():.2e}")
159 if n <= 3000:
160     h = 1e-6 * max(1.0, np.abs(w0[unc]).max()); fd = np.zeros(len(unc))
161     for jj, idx in enumerate(unc):
162         wp = w0.copy(); wp[idx, 0] += h; wm = w0.copy(); wm[idx, 0] -= h
163         fd[jj] = (potential(sampler, i, wp) - potential(sampler, i, wm)) / (2 * h)
164     print(f"gradient: max |adjoint - FD| / max|FD| = {np.abs(g_adj - fd).max() / np.abs(fd).max():.2e}")
165
166 def bench(f, label):
167     f(); t0 = time.time()
168     for _ in range(reps): f()
169     dt = (time.time() - t0) / reps
170     print(f" {label:40s} {dt*1000:9.1f} ms"); return dt
171 print("timings:")
172 t_gref = bench(lambda: gradient_potential_Fourier_domain(sampler, i, w0.copy()), "repo gradient (vectorised over k)")
173 t_gadj = bench(lambda: gradient_potential_adjoint(sampler, i, w0.copy()), "adjoint gradient")
174 t_pref = bench(lambda: potential(sampler, i, w0.copy()), "repo potential")
175 t_padj = bench(lambda: potential_adjoint(sampler, i, w0.copy()), "adjoint potential")
176 L = 10
177 print(f"HMC blur update, L={L}: repo ~{((L+1)*t_gref + 2*t_pref):.2f} s -> adjoint ~{((L+1)*t_gadj + 2*t_padj):.3f} s (
    ↳ speedup {(L+1)*t_gref + 2*t_pref}/((L+1)*t_gadj + 2*t_padj):.0f)x")

```

D Environment and method notes

- Repository: `github.com/guillerminasenn/bsbd`, single commit `c42ccb6` (“initial commit”), cloned 28 Sep 2026. Python 3.11, numpy 2.4.4, scipy 1.17.1 (the repository pins numpy 2.4.1 / scipy 1.16.0; both have the `out=FFT` argument that enables the aliasing bug).
- Hardware: 2-vCPU cloud sandbox, 8 GB RAM. BLAS pinned to one thread (`OPENBLAS_NUM_THREADS=1`) because the default threading produced erratic 30–100 ms stalls on 60×60 operations in this container. Ratios between variants are robust to this; absolute times are not comparable with the paper’s HPC runs.
- All simulated problems use the notebook’s `build_model` path (full well column as exact image observations, endpoint-constrained blur, cyclic topology) with the notebook’s hyper-parameters and seed 20. The paper-size problem uses the paper’s geometry (330×50 observed, margins 150/50, $k = 126$, $m = 330$).
- For the paper-size timing only, unused set-up tensors were skipped to fit in 8 GB (`dW0_Rcv_star`, `bases_dW_Rc`, `fft2_bases_d_W`, `bases_d_W(T)`, `d_W0`); none of them is read by the gradient, potential or Gibbs updates.
- Two minor repository issues hit while running: `MCMC.run` raises when `adapt_config` is `None` (it dereferences the argument after defaulting the attribute), and the gradient raises `KeyError: 'L'` when there are no exact image observations ($m = 0$), although the potential handles that case.
- The derivation in Section 3 was validated end-to-end: the implementation built from it agrees with the (fixed) per-coordinate gradient to 10^{-14} relative and with finite differences to 10^{-7} on all test problems, and chains driven by the two gradients from the same seed are bit-for-bit identical in their accept/reject decisions.